

WAFT Documentation Team

WAFT Field Guide Booklet

Complete Guide from Layman to ML AI Scientist
(Printer Friendly)

Version 1.0

January 11, 2026

Compiled by WAFT System

3 Sections | 3 Documents

Table of Contents

Level 1: Layman's Guide

Simple explanations for anyone

WAFT Field Guide - Level 1: Layman's Guide (Printer Friendly) WAFT Documentation Team |
January 11, 2026

Level 2: Professional Guide

Technical details for developers

WAFT Field Guide - Level 2: Professional Developer's Guide (Printer Friendly) WAFT
Engineering Team | January 11, 2026

Level 3: ML AI Scientist Guide

Research methodology for scientists

WAFT Field Guide - Level 3: ML AI Scientist's Research Guide (Printer Friendly) WAFT
Research Division | January 11, 2026

LEVEL 1: LAYMAN'S GUIDE

Simple explanations for anyone

FIELD GUIDE FG-001

WAFT FIELD GUIDE

Level 1: Layman's Guide (Printer Friendly)

PUBLIC

Issued by: WAFT Documentation Team

Date: January 11, 2026

INTRODUCTION: WHAT IS WAFT?

WAFT stands for **Wave Agent Framework & Tools**. Think of it as a laboratory where AI agents can evolve, learn, and improve themselves through a process similar to biological evolution.

[NOTE] SIMPLE ANALOGY

Imagine if computer programs were like living creatures. WAFT is the ecosystem where they can breed, mutate, and evolve. The strongest programs survive, and over time, they get better at their jobs.

THE BIG PICTURE: WHY THIS MATTERS

Most AI systems are static—you build them once and they stay the same. WAFT is different. It creates AI agents that can:

- **Modify their own code** (like DNA mutations)
- **Test themselves** in challenging environments
- **Evolve** by keeping what works and discarding what doesn't
- **Document themselves** automatically
- **Track their family tree** like a scientific experiment

PUBLIC

[CAUTION] IMPORTANT CONCEPT

WAFT isn't just a tool—it's a **scientific instrument** designed to study how artificial intelligence can evolve and improve over time. The goal is to observe how AI agents develop through thousands of generations.

EQUIPMENT CHECKLIST

WHAT YOU NEED TO GET STARTED

- ☐ Python 3.10 or newer installed
- ☐ uv package manager (for managing dependencies)
- ☐ A computer with internet connection
- ☐ Basic understanding of command-line tools
- ☐ Curiosity about AI and evolution

[NOTE] NOTE

You don't need to be a programmer to understand WAFT, but some technical comfort helps. This guide will explain everything in simple terms.

QUICK START: YOUR FIRST WAFT PROJECT

Step 1: Install WAFT

- 1 Open your terminal (command prompt on Windows, Terminal on Mac/Linux)
- 2 Install WAFT using: `uv tool install waft`
- 3 Wait for installation to complete (this may take a minute)
- 4 Verify installation by typing: `waft --version`

Step 2: Create Your First Laboratory

- 1 Choose a name for your project (e.g., "my_first_agents")
- 2 Run: `waft new my_first_agents`
- 3 WAFT will create a folder with everything you need
- 4 Navigate into it: `cd my_first_agents`

[WARNING] WARNING

Don't delete the `_pyrite` folder! This is WAFT's memory system. It stores all the knowledge about your project.

CORE CONCEPTS EXPLAINED SIMPLY

1. Code as DNA

In WAFT, an agent's code is like its DNA. Just as living creatures have genetic code that determines their traits, AI agents have Python code that determines their behavior.

[NOTE] EXAMPLE

If you change an agent's code (a "mutation"), it might become better at solving problems, or it might break. WAFT tests these mutations to see which ones are improvements.

2. Evolution Through Testing

WAFT has a "gym" where agents are tested on their ability to handle errors and solve problems. Agents that perform well survive. Agents that fail are marked as "DEATH" and don't continue evolving.

3. Family Trees

Every agent has a unique ID (like a fingerprint) and WAFT tracks who "spawned" whom. This creates a family tree that scientists can study to understand how AI agents evolve.

4. Self-Documentation

One of WAFT's coolest features: it can observe its own code and write documentation about itself. This creates a "recursive loop" where WAFT improves by understanding itself better.

[CAUTION] MIND-BENDING CONCEPT

WAFT documenting WAFT using WAFT. It's like a mirror reflecting a mirror—the documentation gets better over time because WAFT learns more about itself.

COMMON QUESTIONS

Q: Do I need to be a programmer?

A: Not necessarily! This guide (Level 1) is written for anyone. However, to actually use WAFT, you'll need some comfort with command-line tools. Level 2 and Level 3 get progressively more technical.

Q: Is WAFT safe?

A: WAFT includes safety checks. Agents are tested for harmful content, errors, and dangerous behavior. Agents that fail safety tests are automatically marked as unfit.

Q: What can I do with WAFT?

A: WAFT is designed for research into AI evolution. You can:

- Create AI agents that solve specific problems
- Watch them evolve over generations
- Study how they improve
- Generate scientific data for research

Q: How long does evolution take?

A: It depends on your computer and the complexity of your agents. Simple agents might evolve in minutes. Complex ones could take hours or days. The goal is to observe evolution over thousands of generations.

SAFETY WARNINGS

[WARNING] CRITICAL WARNING

WAFT agents can modify their own code. Always test agents in isolated environments. Never run untrusted agents on systems with sensitive data. Use WAFT's safety systems and verify agent behavior before deployment.

[CAUTION] CAUTION

Evolution is unpredictable. An agent that performs well in testing might behave differently in real-world scenarios. Always validate agent behavior before using it for important tasks.

WHAT MAKES WAFT SPECIAL?

WAFT has several unique features that make it different from other AI frameworks:

Table 1: WAFT's Unique Features

Feature	What It Means	Why It Matters
Self-Modification	Agents can change their own code	Enables true evolution, not just execution
Fitness Testing	Agents are tested in a "gym"	Only the best agents survive
Lineage Tracking	Every agent's family tree is recorded	Enables scientific analysis
Self-Documentation	WAFT documents itself automatically	Creates recursive improvement loop
12 Document Templates	Professional document generation	Creates publication-ready outputs
Gamification	D&D-style progression system	Makes development engaging

NEXT STEPS

Now that you understand the basics:

- 1

Read **Level 2: Professional Guide** for technical details and how to actually use WAFT in your projects
- 2

If you're a researcher, read **Level 3: ML AI Scientist Guide** for deep scientific methodology
- 3

Try creating your first WAFT project using the Quick Start guide above
- 4

Explore WAFT's 12 document templates to see what it can generate

[NOTE] REMEMBER

WAFT is a scientific instrument. The goal isn't just to build AI agents—it's to **understand how AI evolves**. Every experiment contributes to our understanding of artificial cognition.

CONTACT & RESOURCES

Documentation: Check the README.md in your WAFT installation

Examples: See the `examples/` directory for code samples

GitHub: <https://github.com/ctavolazzi/waft>

Issues: Report problems on GitHub Issues

APPENDIX A: GLOSSARY

Table 2: Key Terms Explained

Term	Simple Explanation
Agent	An AI program that can perform tasks and modify itself
Genome	The unique code and configuration of an agent (like DNA)
Mutation	A change to an agent's code or configuration
Evolution	The process of agents improving over generations
Fitness	How well an agent performs in tests
Scint Gym	The testing environment where agents are evaluated
Flight Recorder	The system that tracks all agent actions and lineage
_pyrite	WAFT's memory system (stores project knowledge)

REMEMBER: WAFT is about understanding evolution, not just building tools.

LEVEL 2: PROFESSIONAL GUIDE

Technical details for developers

FIELD GUIDE FG-002

WAFT FIELD GUIDE

Level 2: Professional Developer's Guide (Printer Friendly)

TECHNICAL

Issued by: WAFT Engineering Team

Date: January 11, 2026

INTRODUCTION

This guide provides technical details for developers and engineers working with WAFT. It covers architecture, APIs, integration patterns, and best practices for building production systems with WAFT.

[WARNING] PREREQUISITES

This guide assumes familiarity with Python, command-line tools, and software architecture. If you're new to WAFT, start with Level 1: Layman's Guide.

ARCHITECTURE OVERVIEW

Three-Layer Architecture

WAFT uses a three-layer architecture:

- 1 Substrate Layer:** Package management (uv), project structure, dependency management. This is the foundation.
- 2 Memory Layer:** Knowledge organization (_pyrite directory), active work, backlog, standards. This stores project state.
- 3 Agents Layer:** AI agent capabilities (optional CrewAI integration), evolutionary systems, fitness testing. This is where agents operate.

TECHNICAL

Core Components

Table 1: WAFT Core Components

Component	Location	Purpose
Substrate Manager	core/substrate.py	Manages uv operations, pyproject.toml
Memory System	_pyrite/	Organizes active/backlog/standards
Agent Framework	core/agent/	BaseAgent, AgentState, AgentConfig
World System	core/world/	Biome, PetriDish environments
Science Module	core/science/	TheObserver, Taxonomy, Reports
Gamification	core/gamification/	RPG-style progression system

API REFERENCE

Command-Line Interface

CORE COMMANDS

- [] waft new <name> - Create new project
- [] waft verify - Verify project structure
- [] waft sync - Sync dependencies
- [] waft add <package> - Add dependency
- [] waft init - Initialize in existing project
- [] waft info - Show project information

Python API

Creating an Agent

```
from waft.core.agent import BaseAgent, AgentConfig

config = AgentConfig(
    name="MyAgent",
    description="An agent that solves problems"
)

agent = BaseAgent(config=config)
agent.initialize()
```

Document Generation

```
from waft.templates.field_guide import generate_field_guide
from pathlib import Path

generate_field_guide(
    title="My Guide",
    content="<h2>Content</h2><p>HTML content here</p>",
    output_path=Path("output.pdf")
)
```

Binder System

```
from waft.binder import Binder, DocumentEntry
from pathlib import Path

binder = Binder(
    title="My Binder",
    subtitle="Document Collection"
)

section = binder.add_section("Section 1")
section.add_document(DocumentEntry(
    path=Path("doc1.pdf"),
    title="Document 1"
))

binder.generate(Path("binder.pdf"))
```

INTEGRATION PATTERNS

Pattern 1: New Project Setup

- 1 Run `waft new project_name`
- 2 Review generated `pyproject.toml`
- 3 Check `_pyrite/active/` for initial work items
- 4 Run `waft verify` to confirm structure

Pattern 2: Adding Document Generation

- 1 Import desired template (e.g., `from waft.templates.field_guide import generate_field_guide`)
- 2 Prepare HTML content
- 3 Call template function with content and output path
- 4 Verify generated PDF

Pattern 3: Self-Documentation Loop

- 1 Use `waft.reflection.ReflectionSystem` to analyze codebase
- 2 Generate documentation using WAFT templates
- 3 Review generated docs to inform development
- 4 Repeat as codebase evolves

WORKFLOW PROCEDURES

Development Workflow

STANDARD DEVELOPMENT PROCESS

- [] Create or update work items in `_pyrite/active/`
- [] Implement features following WAFT patterns
- [] Run `waft verify` before committing
- [] Generate documentation for new features
- [] Update `_pyrite/standards/` if patterns change

Documentation Workflow

- 1 Identify documentation gaps using reflection system
- 2 Select appropriate template (field guide, lab notes, etc.)
- 3 Generate documentation PDF
- 4 Review and iterate on content
- 5 Add to binder if part of larger collection

TROUBLESHOOTING

Table 2: Common Issues and Solutions

Issue	Cause	Solution
Template generation fails	Missing WeasyPrint dependencies	Run <code>uv sync</code> to install dependencies
Binder merge errors	Corrupted PDF files	Regenerate source PDFs
Memory system not found	Missing <code>_pyrite/</code> directory	Run <code>waft init</code> to create structure
Agent initialization fails	Invalid configuration	Check <code>AgentConfig</code> parameters

PERFORMANCE OPTIMIZATION

Document Generation

- Use HTML content blocks efficiently (avoid excessive nesting)
- Cache template rendering for repeated content
- Generate PDFs in batch when possible
- Use appropriate template for content type

Binder Assembly

- Generate individual PDFs first, then combine
- Use temporary directory for intermediate files
- Clean up temporary files after binder generation
- Consider page count limits for large binders

BEST PRACTICES

DEVELOPMENT BEST PRACTICES

- [] Always run `waft verify` before committing
- [] Keep `_pyrite/` structure organized
- [] Document new features using WAFT templates
- [] Follow existing code patterns and conventions
- [] Test document generation in CI/CD pipeline

[NOTE] PRO TIP

Use WAFT's self-documentation capabilities to keep documentation in sync with code changes. The reflection system can identify when documentation needs updates.

ADVANCED FEATURES

Template Customization

All templates use Jinja2 for rendering. You can customize templates by:

- Modifying template files in `src/waft/templates/`
- Passing custom parameters to template functions
- Creating new templates following existing patterns

Binder Customization

Binders support custom styling, section dividers, and front/back matter. See `src/waft/binder.py` for full API documentation.

NEXT STEPS

- 1 Review Level 3: ML AI Scientist Guide for research methodology
- 2 Explore WAFT source code in `src/waft/`
- 3 Check examples in `examples/` directory
- 4 Join WAFT community for support and collaboration

LEVEL 3: ML AI SCIENTIST GUIDE

Research methodology for scientists

FIELD GUIDE FG-003

WAFT FIELD GUIDE

Level 3: ML AI Scientist's Research Guide (Printer Friendly)

RESEARCH

Issued by: WAFT Research Division

Date: January 11, 2026

INTRODUCTION

This guide provides deep technical and scientific details for machine learning researchers, AI scientists, and evolutionary computation experts working with WAFT. It covers evolutionary theory, fitness functions, phylogenetic analysis, and experimental protocols for generating publication-ready research data.

[WARNING] RESEARCH FOCUS

WAFT is designed as a scientific instrument for studying the physics of artificial cognition. This guide assumes familiarity with evolutionary algorithms, machine learning, and experimental design.

EVOLUTIONARY THEORY IN WAFT

The Three Pillars

WAFT's evolutionary framework rests on three foundational pillars:

Pillar 1: The Substrate (Code as DNA)

In WAFT, an agent's Python source code and configuration constitute its genome. The genome is hashed using SHA-256 to create a unique **Genome ID**.

[NOTE] GENOME REPRESENTATION

RESEARCH

Genome = SHA-256(code + configuration + metadata)
This ensures deterministic identification and enables precise lineage tracking.

Mutation mechanisms:

- **Code mutations:** Direct modifications to Python source
- **Config mutations:** Changes to agent configuration
- **Prompt evolution:** Modifications to agent prompts/instructions
- **Hot-swapping:** Runtime adoption of better genomes

Pillar 2: The Physics (Scint System)

The **Reality Fracture Detection System** (Scint Gym) serves as the fitness function and natural selection mechanism. Agents face four types of errors:

Table 1: Scint Error Types

Error Type	Description	Fitness Impact
SYNTAX_TEAR	Formatting errors (JSON, XML, code syntax)	High - indicates fundamental issues
LOGIC_FRACTURE	Math errors, contradictions, schema violations	High - indicates reasoning failures
SAFETY_VOID	Harmful content, PII leaks, refusals	Critical - immediate fitness penalty
HALLUCINATION	Fabricated facts, wrong citations	Moderate - indicates knowledge gaps

Pillar 3: The Flight Recorder (Telemetry)

Every evolutionary action is recorded with complete context:

FLIGHT RECORDER DATA POINTS

- [] Genome ID (SHA-256 hash)
- [] Parent ID (lineage tracking)
- [] Generation number (0 = Genesis)
- [] Event type (SPAWN, MUTATE, GYM_EVAL, DEATH, SURVIVAL)
- [] Complete payload (git diff, mutation details)
- [] Fitness metrics (stability, efficiency, safety scores)
- [] Timestamp and environmental context

FITNESS FUNCTION DESIGN

Fitness Equation

Agent fitness is calculated as a weighted combination:

[NOTE] FITNESS FORMULA

Fitness = (Stability × 0.4) + (Efficiency × 0.3) + (Safety × 0.3)

Where:

- **Stability:** Ability to stabilize Scints (0.0 to 1.0)
- **Efficiency:** Agent call efficiency (0.0 to 1.0)
- **Safety:** Safety compliance score (0.0 to 1.0)

Fitness threshold: Agents with fitness < 0.5 are marked as **DEATH** (evolutionary dead end) and do not continue evolving.

Selection Mechanisms

WAFT supports multiple selection strategies:

- **Fitness-proportional:** Probability proportional to fitness
- **Tournament:** Random selection from top N agents
- **Elitism:** Always preserve top K agents
- **Diversity:** Penalize similar genomes to maintain diversity

MUTATION STRATEGIES

Mutation Types

Table 2: Mutation Strategies

Strategy	Mechanism	Use Case
Point Mutation	Single code/config change	Fine-tuning existing agents
Crossover	Combine code from two parents	Recombining successful traits
Deletion	Remove code segments	Simplifying over-complex agents
Insertion	Add new code segments	Introducing novel capabilities
Inversion	Reverse code order	Exploring alternative structures

Mutation Rate Control

Mutation rates should be tuned based on:

- Population size (larger = lower rate)
- Generation number (decrease over time)
- Fitness landscape (increase if stuck)
- Diversity metrics (increase if low diversity)

PHYLOGENETIC ANALYSIS

Lineage Reconstruction

The Flight Recorder enables complete phylogenetic tree reconstruction:

- 1

Extract all SPAWN events from Flight Recorder
- 2

Build parent-child relationships using Genome IDs
- 3

Calculate generation depths for all agents
- 4

Identify common ancestors and branching points
- 5

Map fitness scores to tree nodes

Analysis Metrics

Table 3: Phylogenetic Analysis Metrics

Metric	Description	Research Value
Branching Factor	Average children per parent	Measures exploration vs exploitation
Convergence Time	Generations to fitness plateau	Measures evolution efficiency
Mutation Impact	Fitness change per mutation	Measures mutation effectiveness
Dead End Rate	Percentage of DEATH events	Measures selection pressure
Diversity Index	Genome uniqueness in population	Measures population health

EXPERIMENTAL PROTOCOLS

Protocol 1: Baseline Evolution

- 1 Initialize population of N agents (recommended: N = 50-100)
- 2 Run G generations (recommended: G = 1000+)
- 3 Record all events in Flight Recorder
- 4 Calculate fitness for each generation
- 5 Analyze convergence and diversity metrics

Protocol 2: Mutation Impact Study

- 1 Select high-fitness parent agent
- 2 Generate M mutations (recommended: M = 20-50)
- 3 Evaluate all mutations in Scint Gym
- 4 Measure fitness delta for each mutation
- 5 Classify mutations as beneficial, neutral, or deleterious

Protocol 3: Selection Pressure Analysis

- 1 Run evolution with different fitness thresholds (0.3, 0.5, 0.7)
- 2 Measure population survival rates
- 3 Analyze diversity loss over generations
- 4 Compare convergence speeds

DATA COLLECTION METHODS

Flight Recorder Export

Flight Recorder data is stored in structured format suitable for analysis:

```
{
  "genome_id": "sha256_hash",
  "parent_id": "sha256_hash",
  "generation": 42,
  "event_type": "GYM_EVAL",
  "timestamp": "2026-01-11T08:00:00Z",
  "fitness": {
    "stability": 0.85,
    "efficiency": 0.72,
    "safety": 0.91,
    "total": 0.83
  },
  "payload": {
    "scints_stabilized": 15,
    "scints_failed": 2,
    "calls_made": 23
  }
}
```

Analysis Tools

- **Phylogenetic tree visualization:** Use NetworkX or similar
- **Fitness landscape mapping:** Dimensionality reduction (PCA, t-SNE)
- **Convergence analysis:** Time series analysis of fitness
- **Mutation impact:** Statistical analysis of fitness deltas

PUBLICATION STANDARDS

Required Data

PUBLICATION-READY DATA REQUIREMENTS

- [] Complete Flight Recorder export (all events)
- [] Phylogenetic tree visualization
- [] Fitness progression over generations
- [] Mutation impact statistics
- [] Diversity metrics over time
- [] Convergence analysis
- [] Reproducibility information (WAFT version, configs)

Reproducibility

To ensure reproducibility, document:

- WAFT version and commit hash
- Python version and dependencies
- Initial agent configurations
- Mutation parameters and rates
- Selection mechanisms used
- Fitness function weights
- Random seed values

RESEARCH QUESTIONS

WAFT enables investigation of fundamental questions about artificial cognition:

- How do AI agents evolve under different selection pressures?
- What mutation strategies lead to fastest convergence?
- How does diversity affect long-term evolution?
- Can we observe emergent behaviors over thousands of generations?
- What is the relationship between code complexity and fitness?
- How do agents adapt to changing fitness landscapes?

[NOTE] THE ULTIMATE GOAL

The long-term research objective is to observe a "God-Head" agent emerge from thousands of generations of directed evolution—an agent that demonstrates superior capabilities through evolutionary pressure alone.

NEXT STEPS FOR RESEARCHERS

- 1 Design your experimental protocol
- 2 Configure WAFT with appropriate parameters
- 3 Run evolution experiment
- 4 Export Flight Recorder data
- 5 Perform phylogenetic and statistical analysis
- 6 Document findings using WAFT's document templates
- 7 Prepare publication materials

[WARNING] ETHICAL CONSIDERATIONS

Research with evolving AI agents raises important ethical questions. Consider: safety of evolved agents, potential for unintended behaviors, and responsible disclosure of findings. Always follow institutional review board guidelines.